

APPENDIX

This section provides supplementary materials, including detailed parameter settings of our method, and the KG schemas generated by different methods.

A. Parameter Setting

In this paper, we implement our method and baselines by calling GPT-4o. It is the latest model of OpenAI, which has outstanding text understanding capabilities and can perform relatively complex inference tasks [1]. When calling the LLM, some parameters usually need to be set, including temperature, max_tokens, n, frequency_penalty, and presence_penalty. Among them, temperature is used to control the randomness of the generated text. To ensure the stability of our method, we set it to 0 so that the LLM can generate more deterministic results. Max_tokens is used to specify the maximum length of the generated result. Since the result lengths output by different units are different, max_tokens has no fixed value. For example, the max_tokens of the entity extraction unit is set to 128; while for the entity type fusion unit and the relation type fusion unit, the max_token is set to 4096. The parameter n represents the number of generated results and is set to 1. In addition, frequency_penalty and presence_penalty are used to control the coherence of the generated text, and they are kept as the default values (0).

B. The KG Schema Generated by Each Method

In this section, we will introduce the KG schema generated by each method. First, Table I presents the 13 relation types generated by our method, 9 of which overlap with the relation types in the existing MKC method [2]. For example, equivalence and function similarity both indicate that entities are similar or equal in functionality. However, we also discover four unique relation types, including:

- **Modification:** It means that One entity alters or modifies another entity. For instance, *remove()* can modify the elements in a *SortedSet*.
- **Execution:** It represents that one entity initiates or carries out the operation of another entity. For example, execute *lock()* to close the *Lock* instance.
- **Access:** It implies that one entity retrieves or acquires data from another entity. For example, *readInt()* reads data from a *DataInputStream*.
- **Limitation:** It signifies that one entity imposes constraints on another entity’s behavior or functionality. For example, the output of *add()* is limited by the state of the *BlockingQueue*.

Table II compares the differences in type information between existing methods and our method. EDC [3] is a schema-free method that focuses only on modeling relation types while neglecting entity types. In contrast, MKC defines 3 entity types (package, class, method) and 11 relation types, while our method defines 4 entity types (package, class, method, interface) and 13 relation types, generating a total of 34 type triples (24 of which are valid). This ensures the richness of the KG. Although the EDC method can refine relation types,

there is still redundancy in the final relation types, which can be further optimized. For example, the relation types such as “checks”, “precedes”, and “test” have similar semantics and can be further merged. In contrast, our method can abstract low-dimensional relation types into high-dimensional ones, avoiding such semantic redundancy.

Table III shows the KG schemas designed by different variant methods. Due to ECV_{w/oKE} adopts the schema of MKC, which only contains 3 entity types and 11 relation types, resulting in 11 type triples. ECV_{w/oKV}’s entity and relations types align with ours, but due to the lack of the KG verification module, it includes 208 type triples, only 29 of which are valid, making the constructed KG unreliable. ECV_{w/oFC}, although consistent with our entity and relation types, lacks a full-connectivity strategy, resulting in only 20 type triples (including 16 correct type triples), making it impossible to construct a comprehensive and rich KG.

REFERENCES

- [1] Anita Kirkovska Akash Sharma, Sidd Seethepalli. Analysis: Gpt-4o vs gpt-4 turbo. <https://www.vellum.ai/blog/analysis-gpt-4o-vs-gpt-4-turbo>. Accessed: 2024.8.
- [2] Qing Huang, Zhiqiang Yuan, Zhenchang Xing, Zhengkang Zuo, Changjing Wang, and Xin Xia. 1+ 1 $\neq$ 2: Programming know-what and know-how knowledge fusion, semantic enrichment and coherent application. *IEEE Transactions on Services Computing*, 16(3):1540–1554, 2022.
- [3] Bowen Zhang and Harold Soh. Extract, define, canonicalize: An llm-based framework for knowledge graph construction. In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2024.

TABLE I: The Details of the Relation Types in Our KG Schema.

Type (type in MKC [2])	Definition	Example
Equivalence (function similarity)	One entity is equal or very similar to another entity in functionality.	The offerLast() method adds an element to the end of the Deque, just like offer().
Difference (behavior difference)	One entity is different from another, typically in behavior or characteristics.	The add() and offer() methods behave differently when the queue is full.
Replacement (function replace)	One entity can substitute another in certain contexts without changing the expected result.	In many cases, you can replace the File class with the Path interface.
Preference (efficiency comparison)	One entity is favored over another due to efficiency or ease of use in a specific context.	BufferedInputStream is faster than reading single bytes from an InputStream...
Dependency (logic constraint)	One entity depends on another for its functionality or operation.	Collections.sort() relies on Arrays.asList() to sort array elements when more complex sorting is required.
Implementation (implement constraint)	One entity provides a concrete realization or behavior for another entity.	The PoolThreadRunnable class implements the Runnable interface, allowing it to be executed by a thread.
Collaboration (function collaboration)	One entity communicates or works with another entity to complete a specific task.	To set a date on a PreparedStatement or get a date from a ResultSet, you interact with java.sql.Date.
Conversion (type conversion)	One entity is transformed into another entity or format.	You can convert a Set to a List by passing the Set to the addAll() method of a new List.
Containment (contain/has method)	One entity contains another entity within it.	The headMap() method of SortedMap returns a new map containing the first elements of the original map.
Modification	One entity alters or modifies another entity.	To remove an element from a SortedSet, you call its remove() method, passing the element to be removed.
Execution	One entity initiates or carries out the operation of another entity.	To lock the Lock instance, you must call its lock() method.
Access	One entity retrieves or acquires data from another entity.	You can read data from a DataInputStream using its readInt() method.
Limitation	One entity imposes constraints on another entity's behavior or functionality.	If the BlockingQueue does not have space for a new element, the add() method throws an IllegalStateException.

TABLE II: Comparison of Type Information between the Existing Method and Our Method

Category	MKC		GraphRAG		EDC		Our	
	Number	Content	Number	Content	Number	Content	Number	Content
Entity Type	3	package, class, method	3	package, class, method	0	-	4	package, class, method, interface
Relation Type	11	efficiency comparison, function collaboration, behavior difference, implement constraint, type conversion, logic constraint, function similarity, function opposite, function replace, ...	0	-	53	checks, precedes, test, inspects, is called before, created inside, provides, located in package, uses, operates on ...	13	collaboration, replacement, difference, implementation, conversion, dependency, equivalence, execution, limitation, containment, access, modification
Type Triple	11	(class, efficiency comparison, class), (class, function collaboration, class), (package, contain, class), (method, behavior difference, method), (method, implement constraint, method), (class, type conversion, class), (method, logic constraint, method), (method, function similarity, method), (method, function opposite, method), (method, function replace, method), (class, has method, method)	9	(class, null, class), (class, null, method), (class, null, package), (method, null, method), (method, null, class), (method, null, package), (package, null, package), (package, null, class), (package, null, method)	53	(null, check, null), (null, precedes, null), (null, test, null), (null, inspects, null), (null, is called before, null), (null, created inside, null), (null, provides, null), (null, located in package, null), (null, uses, null), ...	34	(class, preference, class), (class, collaboration, class), (package, containment, class), (method, difference, method), (class, implementation, class), (class, conversion, class), (method, dependency, interface), (class, equivalence, class), (method, execution, method), (method, limitation, method), (method, replacement, method), (class, access, method), (method, modification, class), (class, difference, class), (method, preference, method), ...

TABLE III: Comparison of KG Schemas between Variant Methods

Category	ECV _{w/ACE}		ECV _{w/OKV}		ECV _{w/DFC}		ECV	
	Number	Content	Number	Content	Number	Content	Number	Content
Entity Type	3	package, class, method	4	package, class, method, interface	4	package, class, method, interface	4	package, class, method, interface
Relation Type	11	efficiency comparison, function collaboration, behavior difference, implement constraint, type conversion, logic constraint, function similarity, function replace, function opposite, contain, has method	13	preference, collaboration, replacement, difference, implementation, conversion, dependency, equivalence, execution, limitation, containment, access, modification	13	collaboration, containment, difference, implementation, conversion, dependency, equivalence, execution, limitation, replacement, access, modification	13	preference, collaboration, containment, difference, implementation, conversion, dependency, equivalence, execution, limitation, replacement, modification, access
Type Triple	11	(class, efficiency comparison, class), (class, function collaboration, class), (package, contain, class), (method, behavior difference, method), (method, implement constraint, method), (class, type conversion, class), (method, logic constraint, method), (method, function similarity, method), (method, function opposite, method), (method, function replace, method), (class, has method, method)	208	(class, preference, class), (class, collaboration, class), (package, containment, class), (method, difference, method), (class, implementation, class), (class, conversion, class), (method, dependency, interface), (class, equivalence, class), (method, execution, method), (method, limitation, method), (method, replacement, method), (class, access, method), (method, modification, class), (class, difference, class), (method, preference, method), ...	20	(class, preference, class), (method, collaboration, method), (package, containment, method), (method, difference, method), (class, implementation, class), (class, conversion, class), (method, dependency, method), (method, equivalence, method), (method, execution, method), (method, limitation, method), (method, replacement, method), (method, modification, class), ...	34	(class, preference, class), (class, collaboration, class), (package, containment, class), (method, difference, method), (class, implementation, class), (class, conversion, class), (method, dependency, interface), (class, equivalence, class), (method, execution, method), (method, limitation, method), (method, replacement, method), (class, access, method), (method, modification, class), (class, difference, class), (method, preference, method), ...